

МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
им. Н.Э. Баумана

Факультет “Информатика и системы управления”
Кафедра “Системы обработки информации и управления”



Дисциплина “Парадигмы и конструкции языков программирования”

Отчет по рубежному контролю № 1

Вариант № 2

Вариант запросов Г

Выполнил:
Студент группы ИУ5-34Б
Яхутль М. А.
Преподаватель:
Гапанюк Ю. Е.

Москва 2025

Задание рубежного контроля

Рубежный контроль представляет собой разработку программы на языке Python, которая выполняет следующие действия:

- 1) Необходимо создать два класса данных в соответствии с Вашим вариантом предметной области, которые связаны отношениями один-ко-многим и многие-ко-многим.
- 2) Пример классов данных для предметной области Сотрудник-Отдел:
 1. Класс «Сотрудник», содержащий поля:
 - ID записи о сотруднике;
 - Фамилия сотрудника;
 - Зарплата (количественный признак);
 - ID записи об отделе. (для реализации связи один-ко-многим)
 2. Класс «Отдел», содержащий поля:
 - ID записи об отделе;
 - Наименование отдела.
 3. (Для реализации связи многие-ко-многим) Класс «Сотрудники отдела», содержащий поля:
 - ID записи о сотруднике;
 - ID записи об отделе.

2) Необходимо создать списки объектов классов, содержащих тестовые данные (3-5 записей), таким образом, чтобы первичные и вторичные ключи соответствующих записей были связаны по идентификаторам.

3) Необходимо разработать запросы в соответствии с Вашим вариантом. Запросы сформулированы в терминах классов «Сотрудник» и «Отдел», которые используются в примере. Вам нужно перенести эти требования в Ваш вариант предметной области. При разработке запросов необходимо по возможности использовать функциональные возможности языка Python (list/dict comprehensions, функции высших порядков).

Для реализации запроса №2 введите в класс, находящийся на стороне связи «много», произвольный количественный признак, например, «зарплата сотрудника».

Результатом рубежного контроля является документ в формате PDF, который содержит текст программы и результаты ее выполнения.

Задание варианта

Классы: «Школьник» и «Класс»

Вариант Г.

1. «Отдел» и «Сотрудник» связаны соотношением один-ко-многим. Выведите список всех отделов, у которых название начинается с буквы «А», и список работающих в них сотрудников.
2. «Отдел» и «Сотрудник» связаны соотношением один-ко-многим. Выведите список отделов с максимальной зарплатой сотрудников в каждом отделе, отсортированный по максимальной зарплате.
3. «Отдел» и «Сотрудник» связаны соотношением многие-ко-многим. Выведите список всех связанных сотрудников и отделов, отсортированный по отделам, сортировка по сотрудникам произвольная.

Код программы

```
1 class Klass: 5 usages
2     def __init__(self, class_id: int, name: str):
3         self.class_id = class_id
4         self.name = name
5
6     def __repr__(self):
7         return f"Klass({self.class_id}, '{self.name}')"
8
9
10 class Schoolchild: 8 usages
11     def __init__(self, child_id: int, name: str, grade: float):
12         self.child_id = child_id
13         self.name = name
14         self.grade = grade
15
16     def __repr__(self):
17         return f"Schoolchild({self.child_id}, '{self.name}', {self.grade})"
18
19
20 class SchoolchildKlass: 8 usages
21     def __init__(self, child_id: int, class_id: int):
22         self.child_id = child_id
23         self.class_id = class_id
24
25     def __repr__(self):
26         return f"SchoolchildKlass(child_id={self.child_id}, class_id={self.class_id})"
27
28
29 classes = [
30     Klass(class_id=1, name="A11"),
31     Klass(class_id=2, name="B10"),
32     Klass(class_id=3, name="A9"),
33     Klass(class_id=4, name="B8"),
34     Klass(class_id=5, name="B7")
35 ]
36
37 schoolchildren = [
38     Schoolchild(child_id=1, name="Иванова Мария", grade=4.5),
39     Schoolchild(child_id=2, name="Петров Алексей", grade=4.8),
40     Schoolchild(child_id=3, name="Сидорова Анна", grade=4.2),
41     Schoolchild(child_id=4, name="Козлов Михаил", grade=4.9),
42     Schoolchild(child_id=5, name="Морозова Елена", grade=4.6),
43     Schoolchild(child_id=6, name="Лебедев Роман", grade=4.3),
44     Schoolchild(child_id=7, name="Федорова Ольга", grade=4.7),
45     Schoolchild(child_id=8, name="Григорьев Денис", grade=4.1),
46 ]
47
48 schoolchild_klasses = [
49     SchoolchildKlass(child_id=1, class_id=1),
50     SchoolchildKlass(child_id=2, class_id=1),
51     SchoolchildKlass(child_id=3, class_id=2),
52     SchoolchildKlass(child_id=4, class_id=2),
53     SchoolchildKlass(child_id=5, class_id=3),
54     SchoolchildKlass(child_id=6, class_id=3),
55     SchoolchildKlass(child_id=7, class_id=4),
56     SchoolchildKlass(child_id=8, class_id=5),
57 ]
58
59 def query_1(): 1 usage
60     print("=== Запрос 1 ===")
61
62     classes_starting_with_A = [k for k in classes if k.name.startswith('A')]
63     print("Классы, начинающиеся с 'A':")
64     for klass in classes_starting_with_A:
65         print(f" - {klass.name}")
66
67     children_in_A_classes = []
68     for j in schoolchild_klasses:
69         if any(k.class_id == j.class_id for k in classes_starting_with_A):
70             child = next(c for c in schoolchildren if c.child_id == j.child_id)
```

```

71         children_in_A_classes.append((child, j.class_id))
72
73     print("\nШкольники в этих классах:")
74     for child, class_id in children_in_A_classes:
75         class_name = next(k.name for k in classes if k.class_id == class_id)
76         print(f" - {child.name} (класс: {class_name})")
77
78 def query_2(): 1 usage
79
80     print("\n=== Запрос 2 ===")
81
82     klass_grades = {}
83     for sk in schoolchild_klasses:
84         class_id = sk.class_id
85         child = next(c for c in schoolchildren if c.child_id == sk.child_id)
86         if class_id not in klass_grades:
87             klass_grades[class_id] = []
88         klass_grades[class_id].append(child.grade)
89
90     klass_avg_grades = {
91         class_id: sum(grades) / len(grades)
92         for class_id, grades in klass_grades.items()
93     }
94
95     sorted_klasses = sorted(
96         klass_avg_grades.items(),
97         key=lambda x: x[1],
98         reverse=True
99     )
100
101     print("Классы, отсортированные по среднему баллу (убывание):")
102     for class_id, avg_grade in sorted_klasses:
103         class_name = next(k.name for k in classes if k.class_id == class_id)
104         print(f" - {class_name}: средний балл = {avg_grade:.2f}")
105
106 def query_3(): 1 usage
107
108     print("\n=== Запрос 3 ===")
109
110     associations = []
111     for sk in schoolchild_klasses:
112         klass = next(k for k in classes if k.class_id == sk.class_id)
113         child = next(c for c in schoolchildren if c.child_id == sk.child_id)
114         associations.append((klass, child))
115
116     associations_sorted = sorted(associations, key=lambda x: x[0].name)
117
118     print("Связи школьников и классов (отсортировано по классам):")
119     for klass, child in associations_sorted:
120         print(f" - Класс: {klass.name}, Школьник: {child.name}")
121
122 if __name__ == "__main__":
123     print("=== Программа: Школьник – Класс ===\n")
124     query_1()
125     query_2()
126     query_3()

```

Результаты выполнения

```
=== Программа: Школьник – Класс ===

=== Запрос 1 ===
Классы, начинающиеся с 'A':
  - A11
  - A9

Школьники в этих классах:
  - Иванова Мария (класс: A11)
  - Петров Алексей (класс: A11)
  - Морозова Елена (класс: A9)
  - Лебедев Роман (класс: A9)

=== Запрос 2 ===
Классы, отсортированные по среднему баллу (убывание):
  - B8: средний балл = 4.70
  - A11: средний балл = 4.65
  - B10: средний балл = 4.55
  - A9: средний балл = 4.45
  - B7: средний балл = 4.10

=== Запрос 3 ===
Связи школьников и классов (отсортировано по классам):
  - Класс: A11, Школьник: Иванова Мария
  - Класс: A11, Школьник: Петров Алексей
  - Класс: A9, Школьник: Морозова Елена
  - Класс: A9, Школьник: Лебедев Роман
  - Класс: B10, Школьник: Сидорова Анна
  - Класс: B10, Школьник: Козлов Михаил
  - Класс: B8, Школьник: Федорова Ольга
  - Класс: B7, Школьник: Григорьев Денис

Process finished with exit code 0
```